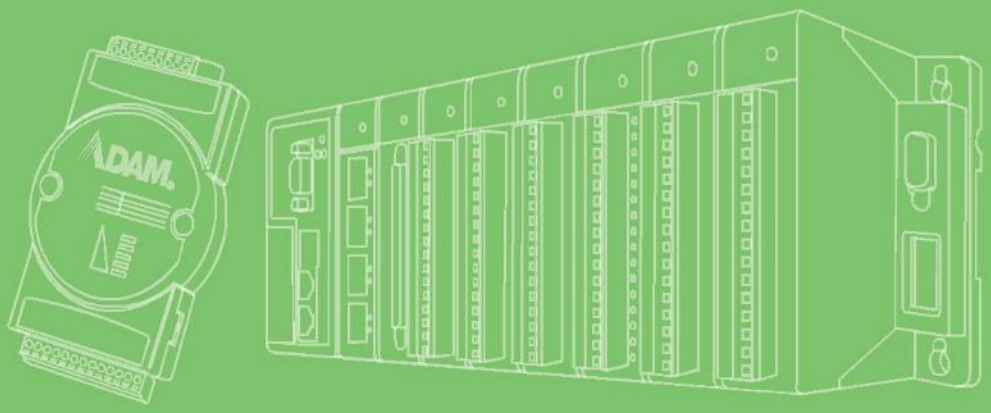


User Manual

Advantech DataConnect Library (Linux)

V1.4

CONTENTS

1. Overview 7

2. Introduction of ADC 8

2.1. Access the symbols 8

2.2. Access the flag memory 10

2.3. Accessing Flag Memory via Symbols 10

2.4. Data connection and sectors 12

2.5. Synchronous Communication Mechanism 14

3. ADC common functions 15

3.1. ADC_Initialize 15

3.2. ADC_Terminate 15

3.3. ADC_GetVersion 16

4. ADC communication functions 17

4.1. ADC_NewDataConnect 17

4.2. ADC_SetMemConnectPort 18

4.3. ADC_ConnectSync 19

4.4. ADC_DisconnectSync 19

4.5. ADC_NewSymdataSector 20

4.6. ADC_AddSymbolToSector 21

4.7. ADC_NewMemdataSector 22

4.8. ADC_GetSymbolList 23

4.8.1. ADC_SYMBOL_INFO 24

4.9. ADC_FreeSymbolList 24

4.10. ADC_ReadSymbolValue 25

4.11. ADC_ReadSymbolValueSync 26

4.12. ADC_WriteSymbolValue 27

4.13. ADC_WriteSymbolValueSync 28

4.14. ADC_ReadSingleMemoryValue 29

4.15. ADC_ReadMultipleMemoryValue 30

4.16. ADC_ReadMemoryValueSync 31

4.17. ADC_WriteSingleMemoryValue 32

4.18. ADC_WriteMultipleMemoryValue 33

4.19. ADC_WriteMemoryValueSync 34

4.20. ADC_StartDataPolling 35

4.21. ADC_StopDataPolling 35

4.22. ADC_SetADCPriority 36

4.23. ADC_ReleaseADCResource 37

4.24. ADC_ShowConnection 37

4.25. ADC_ReadSingleMemoryValue_Local 38

4.26. ADC_ReadMultipleMemoryValue_Local 39

4.27. ADC_WriteSingleMemoryValue_Local 40

4.28. ADC_WriteMultipleMemoryValue_Local 41

5. ADC call back functions 42

5.1. OnConnectCompletedEvent 42

5.2. OnDisconnectCompletedEvent 42
5.3. OnSymdataReadCompletedEvent 43
5.4. OnMemdataReadCompletedEvent 44
5.5. OnSymdataWriteCompletedEvent 44
5.6. OnMemdataWriteCompletedEvent 45
6. Start ADC 46
6.1. Create Modbus TCP Server 46
6.2. adc-package Installation 48
6.2.1. Installation 48
6.2.2. Check ADC Library Version 49
6.2.3. Uninstallation 49
7. ADC calling flow 50
7.1. Asynchronous ADC calling flow 50
7.2. Synchronous ADC calling flow 51
8. ADC sample code 52
8.1. Settings and Build 53
8.1.1. Settings 53
8.1.2. Build Sample Code 54
8.2. Basic example 54
8.3. Access local flag memory 56
8.4. Synchronous access 56
8.5. STRUCT access 57

9. ADC library error code 58

Troubleshooting 61

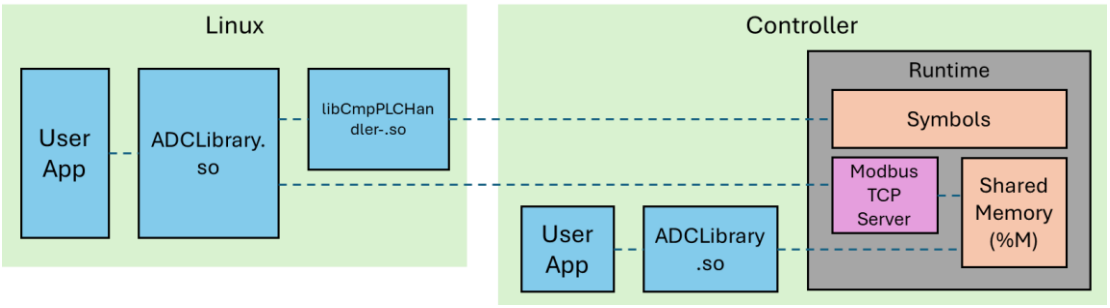
1. PLC channel information 61

Document History:

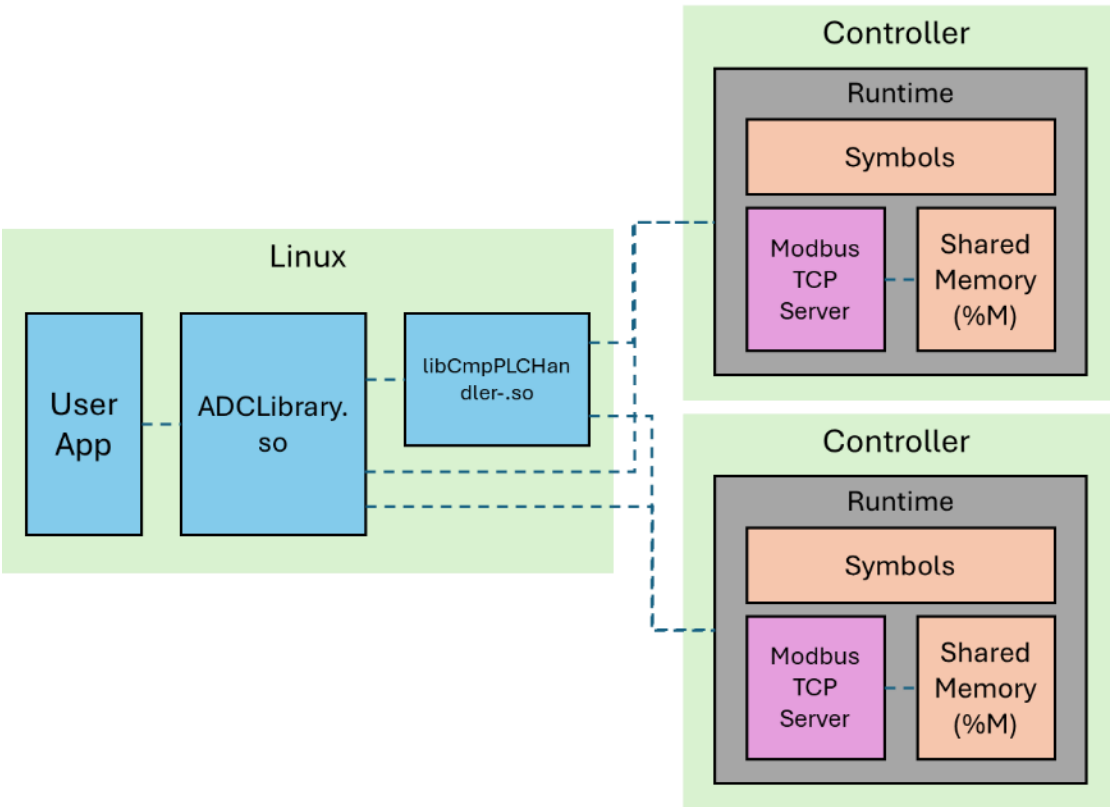
Doc Version	Date	Comment
1.0	2025/12/05	First edition
1.1	2025/12/08	Update the architecture diagram
1.2	2025/12/08	Added supplementary description for asynchronous APIs.
1.3	2026/1/13	Added notes about scan interval limitations and practical behavior. Changed the file path of the example code.
1.4	2026/10/12	Added synchronous connection, Symbol/Memory access, Symbol list, configurable Memory port, error codes and synchronous calling flow for ADC Library 1.0.2.0.

1. Overview

This document describes the Advantech DataConnect Library user manual (Hereinafter referred to as the ADC). ADC is a software interface (API) used for communication between the CODESYS controllers and third-party systems on Linux based system. ADC supports remote access to symbol and flag memory area through the network. And access the local flag memory area at a faster speed (Must be the same device as CODESYS RTE).



ADC also supports that connect to multiple controller

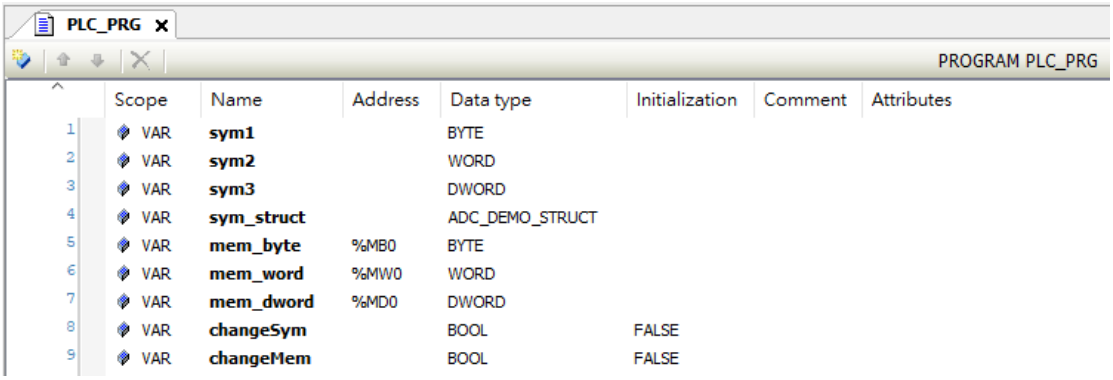


2. Introduction of ADC

Before starting to use ADC library, user need to create CODESYS symbols or define flag memory for variables. And understand the relationship between the ADC connections and sectors.

2.1. Access the symbols

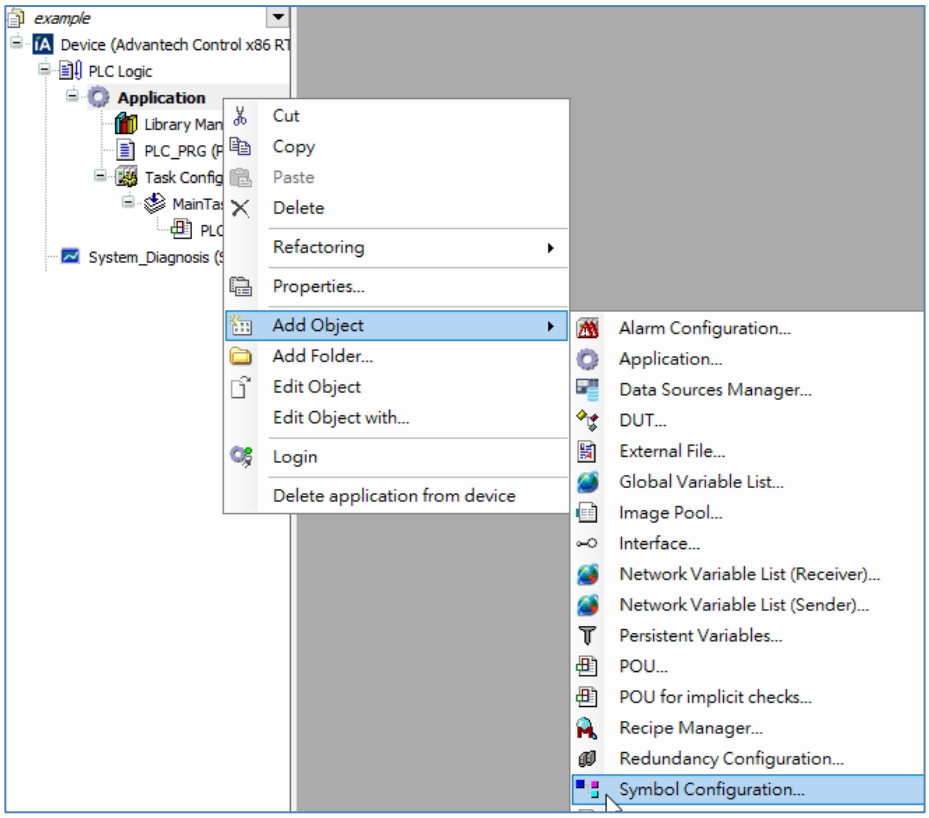
Step1: Create variables



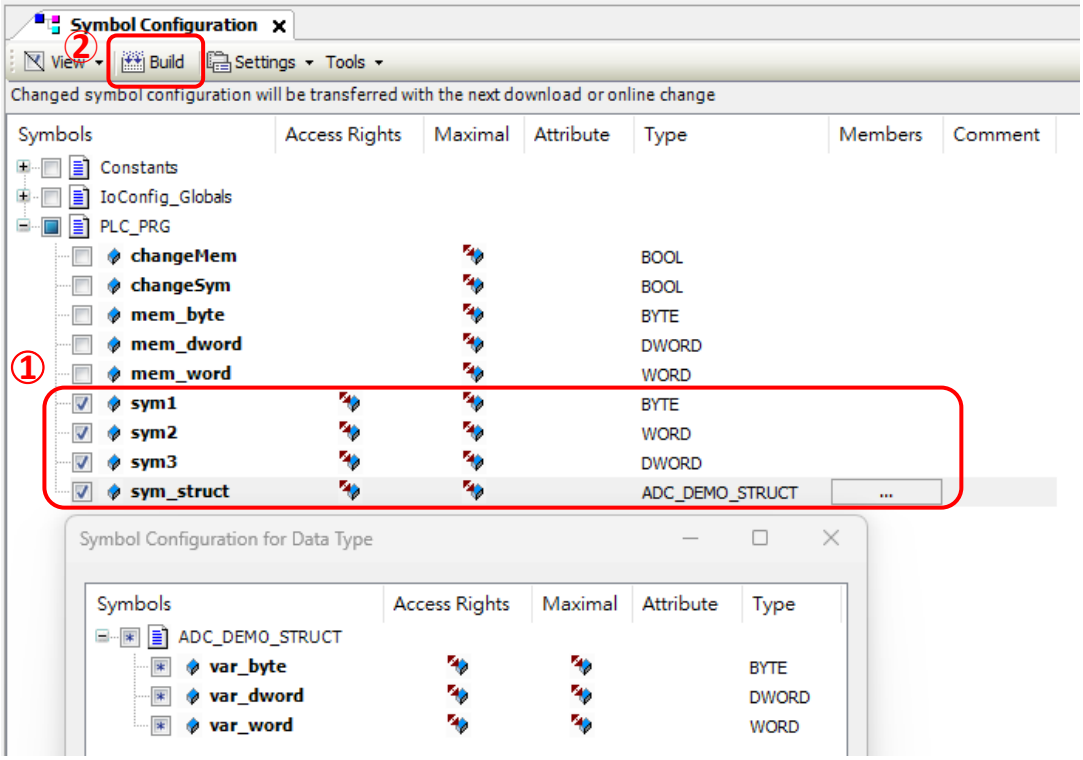
	Scope	Name	Address	Data type	Initialization	Comment	Attributes
1	VAR	**sym1**		BYTE			
2	VAR	**sym2**		WORD			
3	VAR	**sym3**		DWORD			
4	VAR	**sym_struct**		ADC_DEMO_STRUCT			
5	VAR	**mem_byte**	%MB0	BYTE			
6	VAR	**mem_word**	%MW0	WORD			
7	VAR	**mem_dword**	%MD0	DWORD			
8	VAR	**changeSym**		BOOL	FALSE		
9	VAR	**changeMem**		BOOL	FALSE		

Step2: Add Symbol Configuration

Note: The total number of symbols for each data connection is 1000.



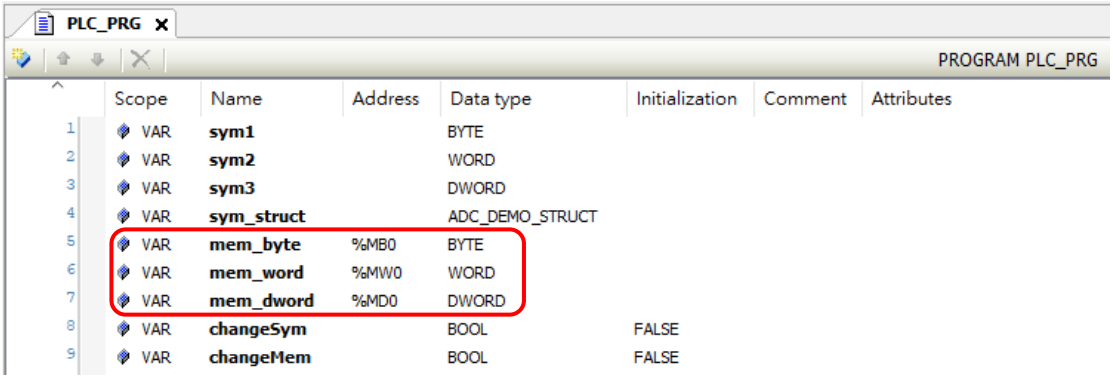
Step3: Select symbols and Build



Step4: Login and download project

2.2. Access the flag memory

Step1: Create variables and map to %M address

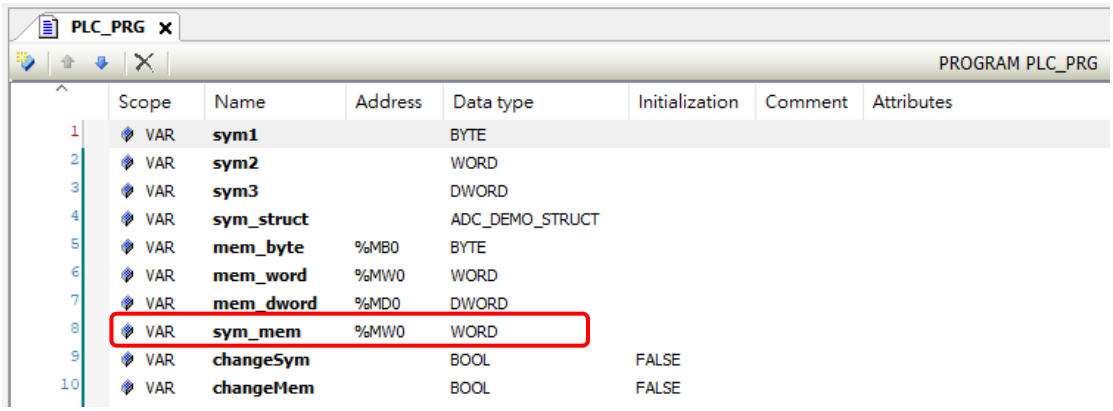


Step2: Login and download project

Note: The total size of the flag memory is 2097152 bytes.

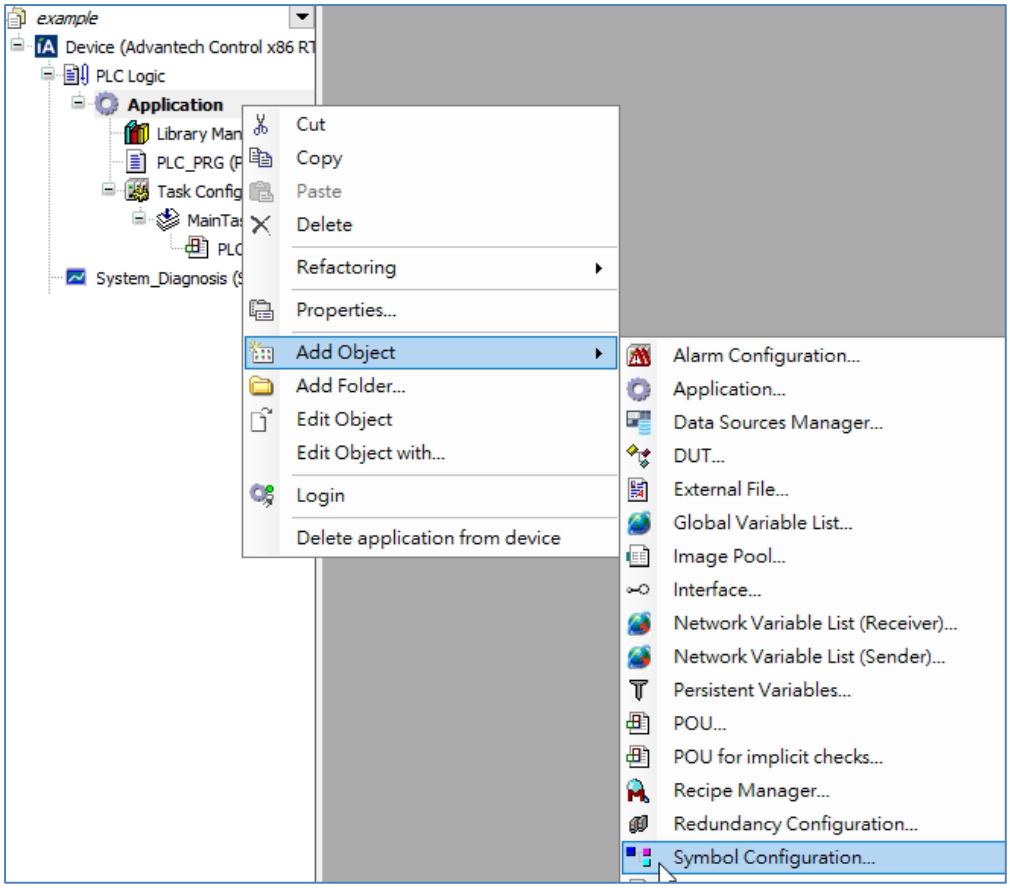
2.3. Accessing Flag Memory via Symbols

Step1: Create variables and map to %M address

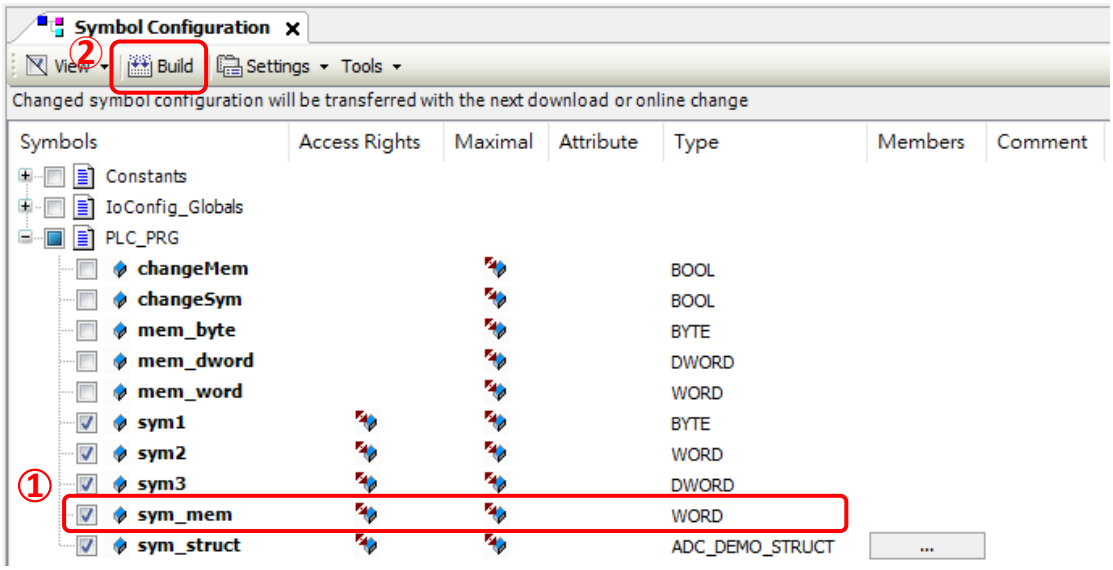


Step2: Add Symbol Configuration

Note: The total number of symbols for each data connection is 1000.



Step3: Select symbols and Build



Step4: Login and download project

2.4. Data connection and sectors

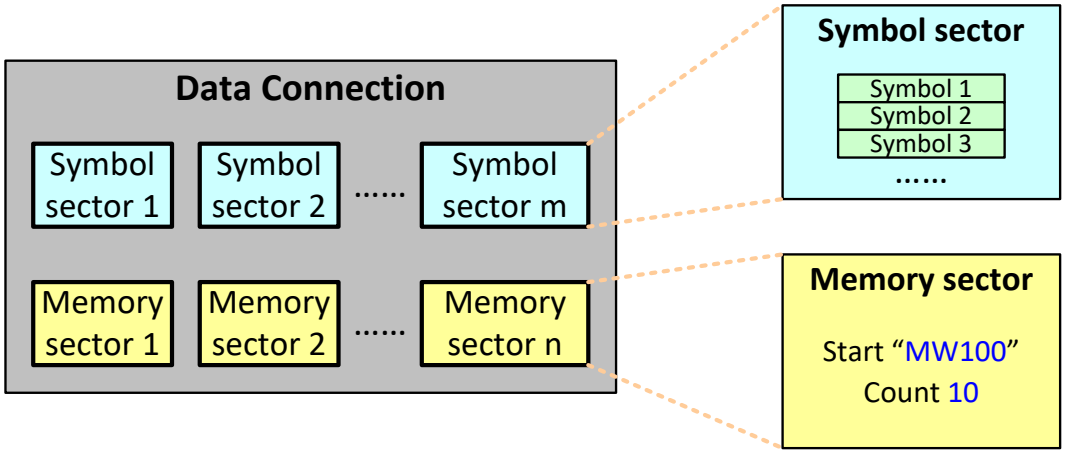
The following diagram describes the relationship between connections and sectors:

User creates a data connection to connect to the controller using the controller IP address. Each data connection contains several symbol sectors and memory sectors. Every symbol sector contains more than one symbol. Every memory sector defines the starting memory address and the total number of %M area (flag memory). After completing the connection and sector creation, call **ADC_StartDataPolling**, the ADC will periodically take values from the controller. User can write values back to controller by calling **ADC_WriteSymbolValue** or **ADC_WriteSingleMemoryValue** (**ADC_WriteMultipleMemoryValue**). The following sections describe the detailed API.

Note: Asynchronous Communication Mechanism

*The ADC library performs symbol and memory access through an **asynchronous polling mechanism**. When ADC_StartDataPolling() is called, a background thread continuously polls all configured connections and caches the symbol and memory data.*

- ***Read APIs** (e.g., ADC_ReadSymbolValue, ADC_ReadSingleMemoryValue) return the latest cached values obtained from the last polling cycle rather than requesting immediate data from the controller.*
- ***Write APIs** (e.g., ADC_WriteSymbolValue, ADC_WriteSingleMemoryValue, ADC_WriteMultipleMemoryValue) queue write requests which are sent by the background thread asynchronously. Their completion is notified through corresponding callback functions (OnSymdataWriteCompletedEvent, OnMemdataWriteCompletedEvent).*
- *A return value of ADC_SUCCESS means the operation request was successfully accepted by the ADC library, **not** that the data has already been applied on the PLC side.*
- *Applications should use the callback functions to confirm completion or handle timing-sensitive logic.*



If you want to access the local flag memory (must be the same device as the CODESYS RTE), you can call read/write APIs (**ADC_ReadSingleMemoryValue_Local** / **ADC_ReadMultipleMemoryValue_Local** / **ADC_WriteSingleMemoryValue_Local** / **ADC_WriteMultipleMemoryValue_Local**) directly without create data connection and sector. And after the initialization is successful, it can be accessed directly without calling **ADC_StartDataPolling**.

2.5. Synchronous Communication Mechanism

For synchronous access, first call ADC_Initialize() and ADC_NewDataConnect() to register connection settings. Optionally call ADC_SetMemConnectPort() before establishing the connection, then call ADC_ConnectSync().

After the connection is established, applications can call ADC_GetSymbolList(), ADC_ReadSymbolValueSync(), ADC_WriteSymbolValueSync(), ADC_ReadMemoryValueSync(), and ADC_WriteMemoryValueSync() directly. Symbol and Memory sectors and ADC_StartDataPolling() are not required for this usage flow.

Call ADC_DisconnectSync(), ADC_ReleaseADCResource(), and ADC_Terminate() when finished. ADC_ConnectSync() establishes both PLCHandler and Modbus TCP connections; both must succeed.

Synchronous Symbol access can share a connected PLCHandler with polling. Synchronous Memory read/write cannot be called while polling is running and return ADC_THREAD_RUNNING in that state. The synchronous Modbus client is managed separately from the polling Modbus client.

Synchronous operations may wait for communication completion or timeout. A write timeout does not guarantee that the controller did not receive the write request.

3. ADC common functions

3.1. ADC_Initialize

long ADS_API ADC_Initialize();

Purpose:

Initialize the ADC library resource.

Parameters:

None.

Return:

ADC_SUCCESS, initialize ADC library succeeded.

ADC_MEM_WSASTART_FAILED, failed to initialize resource.

3.2. ADC_Terminate

long ADS_API ADC_Terminate();

Purpose:

Terminate the ADC library and release resource.

Parameters:

None.

Return:

ADC_SUCCESS, terminate ADC library succeeded.

ADC_MEM_WSACLEAN_FAILED, failed to terminate resource.

3.3. ADC_GetVersion

long ADS_API ADC_GetVersion();

Purpose:

Get the version number of the ADC library.

Parameters:

None.

Return:

The version number. In hexadecimal format, for example 0x03020105, it means the version is V3.2.1.5.

4. ADC communication functions

4.1. ADC_NewDataConnect

```
long ADS_API ADC_NewDataConnect(
        char *i_szConnectName,
        char *i_szIp,
        char *i_szUserName,
        char *i_szPassword,
        int i_iScanInterval,
        int i_iConnectTimeout,
        int i_iTransactTimeout,
        OnConnectCompletedEvent connEvtHandle,
        OnDisconnectCompletedEvent disconnEvtHandle,
        void *i_Param
);
```

Purpose:

Register a data connection and its communication settings. Call this function before ADC_StartDataPolling() for polling access or before ADC_ConnectSync() for explicit synchronous access.

Parameters:

i_szConnectName = The connection name

i_szIp = IP address of the Controller.

i_szUserName = Username for PLC user management.

i_szPassword = Password for PLC user management.

i_iScanInterval = The scan interval for data reading.

Note: The minimum supported scan interval for reading and writing Symbols and Flag Memory is 10 ms. However, the actual read/write execution interval may vary depending on the system performance, operating platform, CPU loading, and network communication conditions. Under heavy system load or unstable network conditions, the effective interval may become longer than the configured scan interval.

i_iConnectTimeout = The connection timeout.

i_iTransactTimeout = The transaction timeout.
connEvtHandle = The pointer of the call back function. The function will be called when connect to Controller completed.
disconnEvtHandle = The pointer of the call back function. The function will be called when disconnect from Controller completed.
i_Param = The pointer of user defined function or object. This pointer will be passed back when callback functions is called.

Return:
ADC_SUCCESS, new the DataConnect succeeded.
ADC_PARAMETER_INVALID, the input parameter is invalid
ADC_THREAD_RUNNING, the polling thread is running.
ADC_OBJECT_DUPLICATED, the connection name is duplicated.

4.2. ADC_SetMemConnectPort

```
long ADS_API ADC_SetMemConnectPort(
        char* i_szConnectName,
        unsigned short i_usPort
);
```

Purpose:
Set the Modbus TCP port for the specified data connection before opening a Memory connection.
Note: Configure the port after ADC_NewDataConnect() and before ADC_ConnectSync() or ADC_StartDataPolling().

Parameters:
i_szConnectName = The connection name previously registered by ADC_NewDataConnect().
i_usPort = Modbus TCP port number (1–65535). The default port is 503.

Return:
ADC_SUCCESS, the port was configured.
ADC_PARAMETER_INVALID, the port or input parameter is invalid.
ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_CONNECTED, the Memory connection is active or polling is running.

4.3. ADC_ConnectSync

long ADS_API ADC_ConnectSync(char* i_szConnectName);

Purpose:
Establish the PLCHandler and synchronous Modbus TCP connections for the specified data connection.
Note: If either connection fails, the partially established synchronous connection is released.
Note: No Symbol or Memory sector is needed for synchronous access.

Parameters:
i_szConnectName = The connection name registered by ADC_NewDataConnect().

Return:
ADC_SUCCESS, both connections have been established.
ADC_HANDLE_INVALID, the connection name is invalid or does not exist.
ADC_PLC_NOT_CONNECTED, the PLCHandler connection could not be established.
ADC_MEM_CONNECT_FAILED, the synchronous Modbus TCP connection could not be established.
Other ADC PLCHandler or Memory communication error codes may be returned.

4.4. ADC_DisconnectSync

long ADS_API ADC_DisconnectSync(char* i_szConnectName);

Purpose:
Release the synchronous connection for the specified data connection.
Note: If polling is using the PLCHandler connection, the shared PLCHandler remains available until polling no longer needs it.

Parameters:
i_szConnectName = The connection name registered by ADC_NewDataConnect().

Return:

ADC_SUCCESS, the synchronous connection has been released.

ADC_HANDLE_INVALID, the connection name is invalid or does not exist.

Other connection-related errors may be returned.

4.5. ADC_NewSymdataSector

```
long ADS_API ADC_NewSymdataSector(
        char *i_szConnectName,
        char* i_szSectorName,
        OnSymdataReadCompletedEvent evtHandle
);
```

Purpose:

New a Symdata sector into an existing connection for polling data from the controller. This function has to be called before calling ADC_StartDataPolling().

Parameters:

i_szConnectName = The connection name

i_szSectorName = The polling sector name.

evtHandle = The pointer of the call back function. The function will be called when reading symbol data completed.

Return:

ADC_SUCCESS, add the sector succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_RUNNING, the polling thread is running.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_DUPLICATED, the sector name is duplicated.

4.6. ADC_AddSymbolToSector

```
long ADS_API ADC_AddSymbolToSector(
        char *i_szConnectName,
        char* i_szSectorName,
        char* i_szSymbolName
);
```

Purpose:

Add a symbol name into an existing sector. This function has to be called before calling ADC_StartDataPolling().

Parameters:

i_szConnectName = The connection name

i_szSectorName = The sector name.

i_szSymbolName = The symbol name.

Return:

ADC_SUCCESS, add the symbol succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_RUNNING, the polling thread is running.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the sector does not exist.

ADC_OBJECT_DUPLICATED, the symbol name is duplicated.

4.7. ADC_NewMemdataSector

```
long ADS_API ADC_NewMemdataSector (
        char *i_szConnectName,
        char* i_szSectorName,
        char* i_szMemStartAddress,
        int i_iTotal,
        OnMemdataReadCompletedEvent evtHandle
);
```

Purpose:

New a memory data sector into the polling list for polling data from the controller. This function has to be called before calling ADC_StartDataPolling().

Parameters:

i_szConnectName = The connection name.

i_szSectorName = The polling sector name.

i_szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item. The maximum number of %MB items in a sector is **253**. The maximum number of %MW items in a sector is **127**. The maximum number of %MD items in a sector is **63**.

evtHandle = The pointer of the call back function. The function will be called when reading memory data completed.

Return:

ADC_SUCCESS, add the sector succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_RUNNING, the polling thread is running.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_DUPLICATED, the sector name is duplicated.

ADC_EXCEED_SECTOR_SIZE, the total size of the memory item exceeds the sector size.

4.8. ADC_GetSymbolList

```
long ADS_API ADC_GetSymbolList(
        char* i_szConnectName,
        ADC_SYMBOL_INFO** o_ppSymbolList,
        unsigned long* o_pSymbolCount
);
```

Purpose:

Retrieve the available PLC Symbol information as an allocated array of ADC_SYMBOL_INFO.

Note: The caller must release the returned array using ADC_FreeSymbolList().

Note: The Symbol names, type names and sizes are copied from PLCHandler metadata.

Parameters:

i_szConnectName = The connection name.

o_ppSymbolList = Output pointer receiving the allocated Symbol information array.

o_pSymbolCount = Output pointer receiving the number of Symbols.

Return:

ADC_SUCCESS, the Symbol list was retrieved. An empty list may be returned as NULL with count 0.

ADC_PARAMETER_INVALID, an input or output argument is invalid.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_PLC_NOT_CONNECTED, the PLCHandler connection is unavailable.

ADC_PLC_SYMBOL_LOAD_FAILED, loading Symbols failed.

ADC_MEMALLOC_FAILED, allocation failed.

Other PLCHandler-related ADC errors may be returned.

4.8.1. ADC_SYMBOL_INFO

```
#define ADC_SYMBOL_NAME_MAX_LENGTH 256
#define ADC_SYMBOL_TYPE_MAX_LENGTH 256
typedef struct tagADC_SYMBOL_INFO
{
    char szName[ADC_SYMBOL_NAME_MAX_LENGTH];
    char szType[ADC_SYMBOL_TYPE_MAX_LENGTH];
    unsigned long ulSize;
} ADC_SYMBOL_INFO;
```

Members:
szName = The Symbol name.
szType = The Symbol type name.
ulSize = The Symbol size in bytes.

Note: The structure occupies 516 bytes on Linux x86 and 520 bytes on Linux x64 because unsigned long is platform-dependent.

4.9. ADC_FreeSymbolList

```
void ADS_API ADC_FreeSymbolList(ADC_SYMBOL_INFO* i_pSymbolList);
```

Purpose:
Release the Symbol information array allocated by ADC_GetSymbolList().
Note: Do not call free() directly or release the same pointer more than once.

Parameters:
i_pSymbolList = The Symbol array pointer returned by ADC_GetSymbolList().

Return:
None (void).

4.10. ADC_ReadSymbolValue

```
long ADS_API ADC_ReadSymbolValue (
        char* i_szConnectName,
        char* i_szSymbolName,
        void* o_pValue,
        int* pLength
);
```

Purpose:

Read the specified symbol value from the connection. This function has to be called after calling ADC_StartDataPolling().

Note: This function is part of the asynchronous data polling system. The ADC background thread manages symbol/memory read and write operations. The return value indicates whether the request was accepted, not whether the operation has completed.

Parameters:

i_szConnectName = The connection name.

i_szSymbolName = The symbol name.

o_pValue = Pointer to the variable of the symbol to be read.

pLength = The length of the declared variable size. The input value must be greater than or equal to the symbol size. After retrieving the symbol value, the output value is expressed as the actual size of the symbol.

Return:

ADC_SUCCESS, read the symbol value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the symbol is not created in the sector.

ADC_PLC_SYMBOL_VALUE_NOT_READY, the value has not been retrieved

ADC_PLC_SYMBOL_SIZE_NOT_MATCH, the variable size is less than the symbol size.

ADC_PLC_NOT_CONNECTED, PLC is not connected.

ADC_PLC_NO_SYMBOLS, no symbols are available or symbol mismatch because of change project.

4.11. ADC_ReadSymbolValueSync

```
long ADS_API ADC_ReadSymbolValueSync(
        char* i_szConnectName,
        char* i_szSymbolName,
        void* o_pValue,
        unsigned long i_ulBufferSize
);
```

Purpose:
Read the specified Symbol value from the PLC and return the result after the read operation completes.

Parameters:
i_szConnectName = The connection name.
i_szSymbolName = The Symbol name.
o_pValue = Output buffer receiving the Symbol value.
i_ulBufferSize = Size of the output buffer, in bytes. It must be at least the actual Symbol size.

Return:
ADC_SUCCESS, the complete Symbol value was read.
ADC_PARAMETER_INVALID, an input parameter is invalid.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.
ADC_OBJECT_NOT_EXIST, the requested Symbol does not exist.
ADC_BUFFER_TOO_SMALL, the output buffer is smaller than the Symbol size.
ADC_INCOMPLETE_DATA, the returned value is incomplete.
ADC_WRONG_CONTEXT, the API is called from a prohibited communication thread.
Other PLCHandler-related ADC error codes may be returned.

4.12. ADC_WriteSymbolValue

```
long ADS_API ADC_WriteSymbolValue (
        char* i_szConnectName,
        char* i_szSymbolName,
        void* i_pValue,
        OnSymdataWriteCompletedEvent evtHandle
);
```

Purpose:

Write the specified symbol value to the controller. The symbol to be written can be written successfully without adding to the sector. This function has to be called after calling ADC_StartDataPolling().

Note: This function is part of the asynchronous data polling system. The ADC background thread manages symbol/memory read and write operations. The return value indicates whether the request was accepted, not whether the operation has completed.

Parameters:

i_szConnectName = The connection name.

i_szSymbolName = The symbol name.

i_pValue = Pointer to the variable of the symbol to be write.

evtHandle = The pointer of the call back function. The function will be called when writing symbol data completed.

Return:

ADC_SUCCESS, write the symbol value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_OBJECT_NOT_EXIST, the symbol does not exist.

ADC_PLC_NOT_CONNECTED, PLC is not connected.

ADC_PLC_NO_SYMBOLS, no symbols are available or symbol mismatch because of change project.

4.13. ADC_WriteSymbolValueSync

```
long ADS_API ADC_WriteSymbolValueSync(
        char* i_szConnectName,
        char* i_szSymbolName,
        const void* i_pValue,
        unsigned long i_ulDataSize
);
```

Purpose:
Write the specified Symbol value to the PLC and return the result after the write operation completes.

Parameters:
i_szConnectName = The connection name.
i_szSymbolName = The Symbol name.
i_pValue = Pointer to the value to write.
i_ulDataSize = Input data size, in bytes. It must match the Symbol size exactly.

Return:
ADC_SUCCESS, the write operation completed successfully.
ADC_PARAMETER_INVALID, an input parameter is invalid.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_PLC_NOT_CONNECTED, the PLCHandler is not connected.
ADC_OBJECT_NOT_EXIST, the requested Symbol does not exist.
ADC_SIZE_MISMATCH, the input data size does not match the Symbol size.
ADC_WRONG_CONTEXT, the API is called from a prohibited communication thread.
Other PLCHandler-related ADC error codes may be returned.

4.14. ADC_ReadSingleMemoryValue

```
long ADS_API ADC_ReadSingleMemoryValue (
        char* i_szConnectName,
        char* i_szMemStartAddress,
        void* o_pValue,
        int* pLength
);
```

Purpose:

Read the specified memory data sector value from the connection. This function has to be called after calling ADC_StartDataPolling().

Note: This function is part of the asynchronous data polling system. The ADC background thread manages symbol/memory read and write operations. The return value indicates whether the request was accepted, not whether the operation has completed.

Parameters:

i_szConnectName = The connection name.

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

o_pValue = Pointer to the variable of the memory data to be read.

pLength = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW”, the variable size is 2 (bytes). After retrieving the memory data value, the output value is expressed as the actual size of the memory data.

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the connection does not connect.

ADC_OBJECT_NOT_EXIST, the memory data is not created in the sector.

4.15. ADC_ReadMultipleMemoryValue

```
long ADS_API ADC_ReadMultipleMemoryValue (
        char* i_szConnectName,
        char* i_szMemStartAddress,
        int i_iTotal,
        void* o_pValue,
        int* pLength
);
```

Purpose:

Read the specified memory data sector value from the connection. This function has to be called after calling ADC_StartDataPolling().

Note: This function is part of the asynchronous data polling system. The ADC background thread manages symbol/memory read and write operations. The return value indicates whether the request was accepted, not whether the operation has completed.

Parameters:

i_szConnectName = The connection name.

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item.

o_pValue = Pointer to the variable of the memory data to be read.

pLength = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW” and the total count is 5, the variable size is 2*5 (bytes). After retrieving the memory data value, the output value is expressed as the actual size of the memory data.

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the connection does not connect.

ADC_OBJECT_NOT_EXIST, the memory data is not created in the sector.

4.16. ADC_ReadMemoryValueSync

```
long ADS_API ADC_ReadMemoryValueSync(
        char* i_szConnectName,
        char* i_szMemStartAddress,
        void* o_pValue,
        unsigned long i_ulDataSize
);
```

Purpose:
Read the specified number of bytes from the controller Memory and return when the communication operation completes.

Parameters:
i_szConnectName = The connection name.
i_szMemStartAddress = The starting CODESYS Memory address, such as %MB0, %MW0, %MD0 or %ML0.
o_pValue = Output buffer receiving the Memory data.
i_ulDataSize = The requested data size in bytes.

Note: Memory addresses use %MBn (1 byte), %MWn (2 bytes), %MDn (4 bytes), and %MLn (8 bytes). The size parameter is always in bytes.
Note: Requests may be split according to protocol and Memory-bank boundaries. If a later segment fails, earlier segments may already have been copied into the output buffer.

Return:
ADC_SUCCESS, the requested Memory data was read.
ADC_PARAMETER_INVALID, the address, range or another parameter is invalid.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_THREAD_RUNNING, Memory polling is running.
ADC_WRONG_CONTEXT, the API is called from the Modbus communication thread.
ADC_MEM_CONNECT_FAILED, the synchronous Modbus connection is unavailable.
ADC_MEM_WAIT_TIMEOUT, a socket wait timed out.
ADC_INCOMPLETE_DATA, the response does not contain the complete required data.
Other Memory transport or Modbus exception codes may be returned.

4.17. ADC_WriteSingleMemoryValue

```
long ADS_API ADC_WriteSingleMemoryValue (
        char* i_szConnectName,
        char* i_szMemStartAddress,
        void* i_pValue,
        int* i_Length,
        OnMemdataWriteCompletedEvent evtHandle
);
```

Purpose:

Write the value to the specified memory area to the controller. The memory area to be written can be written successfully without adding to the sector. This function has to be called after calling ADC_StartDataPolling().

Note: This function is part of the asynchronous data polling system. The ADC background thread manages symbol/memory read and write operations. The return value indicates whether the request was accepted, not whether the operation has completed.

Parameters:

i_szConnectName = The connection name.

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_pValue = Pointer to the variable of the memory data to be write.

i_Length = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW”, the variable size is 2 (bytes).

evtHandle = The pointer of the call back function. The function will be called when writing memory data completed.

Return:

ADC_SUCCESS, write the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the connection does not connect.

ADC_MEM_THREAD_BUSY, the previous write memory operation was not completed.

4.18. ADC_WriteMultipleMemoryValue

```
long ADS_API ADC_WriteMultipleMemoryValue (
        char* i_szConnectName,
        char* i_szMemStartAddress,
        int i_iTotal,
        void* i_pValue,
        int* i_Length,
        OnMemdataWriteCompletedEvent evtHandle
);
```

Purpose:

Write the value to the specified memory area to the controller. The memory area to be written can be written successfully without adding to the sector. This function has to be called after calling ADC_StartDataPolling().

Note: This function is part of the asynchronous data polling system. The ADC background thread manages symbol/memory read and write operations. The return value indicates whether the request was accepted, not whether the operation has completed.

Parameters:

i_szConnectName = The connection name.

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item. The maximum number of %MB items is **253**. The maximum number of %MW items is **127**. The maximum number of %MD items is **63**.

i_pValue = Pointer to the variable of the memory data to be write.

i_Length = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW” and the total count is 5, the variable size is 2*5 (bytes).

evtHandle = The pointer of the call back function. The function will be called when writing memory data completed.

Return:

ADC_SUCCESS, write the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_THREAD_STOP, the polling thread is stop.

ADC_HANDLE_INVALID, the connection does not exist.

ADC_MEM_SOCKET_CLOSED, the connection does not connect.

ADC_MEM_THREAD_BUSY, the previous write memory operation was not completed.

ADC_EXCEED_SECTOR_SIZE, the total size of the memory item exceeds the sector size.

4.19. ADC_WriteMemoryValueSync

```
long ADS_API ADC_WriteMemoryValueSync(
        char* i_szConnectName,
        char* i_szMemStartAddress,
        const void* i_pValue,
        unsigned long i_ulDataSize
);
```

Purpose:

Write the specified number of bytes to controller Memory and return when the communication operation completes.

Parameters:

i_szConnectName = The connection name.

i_szMemStartAddress = The starting CODESYS Memory address, such as %MB0, %MW0, %MD0 or %ML0.

i_pValue = Pointer to the data to write.

i_ulDataSize = The number of bytes to write.

Note: Memory addresses use %MBn (1 byte), %MWn (2 bytes), %MDn (4 bytes), and %MLn (8 bytes). The size parameter is always in bytes.

Note: Requests may be split according to protocol and Memory-bank boundaries. On failure, earlier segments may already have been written.

Return:
ADC_SUCCESS, the write operation completed successfully.
ADC_PARAMETER_INVALID, the address, range or another parameter is invalid.
ADC_HANDLE_INVALID, the connection does not exist.
ADC_THREAD_RUNNING, Memory polling is running.
ADC_WRONG_CONTEXT, the API is called from the Modbus communication thread.
ADC_MEM_CONNECT_FAILED, the synchronous Modbus connection is unavailable.
ADC_MEM_WAIT_TIMEOUT, a socket wait timed out.
Other Memory transport or Modbus exception codes may be returned.

4.20. ADC_StartDataPolling

long ADS_API ADC_StartDataPolling();

Purpose:
Start the connections. After calling with ADC_SUCCESS returned, the ADC threads will start polling all controllers and their symbols/memory data.

Parameters:
None

Return:
ADC_SUCCESS, start all the DataConnect succeeded.
ADC_THREAD_RUNNING, the connections are already started
ADC_CREATE_THREAD_FAILED, failed to create the connection.

4.21. ADC_StopDataPolling

long ADS_API ADC_StopDataPolling();

Purpose:
Stop the connections. After calling this function, the ADC thread will terminate and all polling will stop.

Parameters:

None

Return:

ADC_SUCCESS, stop all the DataConnect succeeded.

4.22. ADC_SetADCPriority

```
long ADS_API ADC_SetADCPriority(
        int iPriority
);
```

Purpose:

Set the priority of the ADC connection running thread.

Before calling this function, the data connection has to be successfully started.

Note: In Linux environments, adjusting thread priorities requires root privileges. If the current user is not root, the system will use the default thread priority.

Parameters:

iPriority = Specifies the priority value for the ADC thread.

This parameter can be one of the following values

THREAD_PRIORITY_TIME_CRITICAL indicates 3 points above normal priority.

THREAD_PRIORITY_HIGHEST indicates 2 points above normal priority.

THREAD_PRIORITY_ABOVE_NORMAL indicates 1 point above normal priority.

THREAD_PRIORITY_NORMAL indicates normal priority.

THREAD_PRIORITY_BELOW_NORMAL indicates 1 point below normal priority.

THREAD_PRIORITY_LOWEST indicates 2 points below normal priority.

THREAD_PRIORITY_IDLE indicates 3 points below normal priority.

Return:

ADC_SUCCESS, set the ADC thread priority succeeded.

ADC_SETPRIO_FAILED, failed to set the thread priority.

4.23. ADC_ReleaseADCResource

long ADS_API ADC_ReleaseADCResource();

Purpose:

Release all the allocated memory for ADC. This function has to be called after calling ADC_StopDataPolling

Parameters:

None

Return:

ADC_SUCCESS, release the ADC resource succeeded.

4.24. ADC_ShowConnection

long ADS_API ADC_ShowConnection ();

Purpose:

Show the current connections configuration.

Parameters:

None

Return:

ADC_SUCCESS, show the configurations succeeded.

4.25. ADC_ReadSingleMemoryValue_Local

```
ADS_API long ADC_ReadSingleMemoryValue_Local (
        char* i_szMemStartAddress,
        void* o_pValue,
        int* pLength
);
```

Purpose:

Read the specified memory data sector value from local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Note: In Linux environments, accessing local memory through APIs requires root privileges.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

o_pValue = Pointer to the variable of the memory data to be read.

pLength = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW”, the variable size is 2 (bytes). After retrieving the memory data value, the output value is expressed as the actual size of the memory data.

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_HANDLE_INVALID, failed to open local flag memory.

4.26. ADC_ReadMultipleMemoryValue_Local

```
ADS_API long ADC_ReadMultipleMemoryValue_Local (
        char* i_szMemStartAddress,
        int i_iTotal,
        void* o_pValue,
        int* pLength
);
```

Purpose:

Read the specified memory data sector value from local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Note: In Linux environments, accessing local memory through APIs requires root privileges.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item.

o_pValue = Pointer to the variable of the memory data to be read.

pLength = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW”, the variable size is 2 (bytes). After retrieving the memory data value, the output value is expressed as the actual size of the memory data.

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_HANDLE_INVALID, failed to open local flag memory.

4.27. ADC_WriteSingleMemoryValue_Local

```
ADS_API long ADC_WriteSingleMemoryValue_Local (
        char* i_szMemStartAddress,
        void* i_pValue,
        int* i_Length
);
```

Purpose:

Write the specified memory data to local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Note: In Linux environments, accessing local memory through APIs requires root privileges.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_pValue = Pointer to the variable of the memory data to be write.

i_Length = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW”, the variable size is 2 (bytes).

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_HANDLE_INVALID, failed to open local flag memory.

4.28. ADC_WriteMultipleMemoryValue_Local

```
ADS_API long ADC_WriteMultipleMemoryValue_Local (
        char* i_szMemStartAddress,
        int i_iTotal,
        void* i_pValue,
        int* i_Length
);
```

Purpose:

Write the specified memory data to local flag memory. This function is only valid on the controller with Advantech CODESYS RTE installed.

Note: In Linux environments, accessing local memory through APIs requires root privileges.

Parameters:

i_ szMemStartAddress = The memory starting address. For example, “%MW100”. Now we support “%MB”, “%MW” and “%MD” memory access.

i_iTotal = The total of the memory item.

i_pValue = Pointer to the variable of the memory data to be write.

i_Length = Pointer to the length of the declared variable size. The input value must be greater than or equal to the memory data size. For example, if the start address is “%MW”, the variable size is 2 (bytes).

Return:

ADC_SUCCESS, read the memory data value succeeded.

ADC_PARAMETER_INVALID, the input parameter is invalid

ADC_HANDLE_INVALID, failed to open local flag memory.

5. ADC call back functions

5.1. OnConnectCompletedEvent

```
typedef void (*OnConnectCompletedEvent)(
        long i_lResult,.
        char *i_szConnectionName,
        void *i_Param
);
```

Purpose:

The prototype of call back function when the connection to the Controller completed event occurred.

Parameters:

i_lResult = The result of the connection.

i_szConnectionName = The connection name

i_Param = The pointer of user defined function or object that passed in by calling ADC_NewDataConnect.

Return:

None

5.2. OnDisconnectCompletedEvent

```
typedef void (*OnDisconnectCompletedEvent)(
        long i_lResult,
        char *i_szConnectionName,
        void *i_Param
);
```

Purpose:

The prototype of call back function when the connection to the Controller closed event occurred.

Parameters:

i_lResult = The result of the connection.

i_szConnectionName = The connection name

i_Param = The pointer of user defined function or object that passed in by calling ADC_NewDataConnect.

Return:

None

5.3. OnSymdataReadCompletedEvent

```
typedef void (*OnSymdataReadCompletedEvent)(
        long i_lResult,
        char *i_szSectorName,
        int i_iTotal,
        void *i_Param
);
```

Purpose:

The prototype of call back function when the symbol data read completed event occurred.

Parameters:

i_lResult = The result of the reading.

i_ szSectorName = The polling sector name.

i_iTotal = The total of the symbol read.

i_Param = The pointer of user defined function or object that passed in by calling ADC_NewDataConnect.

Return:

None

5.4. OnMemdataReadCompletedEvent

```
typedef void (*OnMemdataReadCompletedEvent)(
        long i_lResult,
        char *i_szSectorName,
        int i_iTotal,
        void *i_Param
);
```

Purpose:

The prototype of call back function when the memory data read completed event occurred.

Parameters:

i_lResult = The result of the reading.

i_szSectorName = The polling sector name.

i_iTotal = The total of the memory item read.

i_Param = The pointer of user defined function or object that passed in by calling ADC_NewDataConnect.

Return:

None

5.5. OnSymdataWriteCompletedEvent

```
typedef void (*OnSymdataWriteCompletedEvent)(
        long i_lResult,
        void *i_Param
);
```

Purpose:

The prototype of call back function when the symbol data write completed event occurred.

Parameters:

i_lResult = The result of the connection.
i_Param = The pointer of user defined function or object that passed in by calling ADC_NewDataConnect.

Return:
None

5.6. OnMemdataWriteCompletedEvent

```
typedef void (*OnMemdataWriteCompletedEvent)(
        long i_lResult,
        void *i_Param
);
```

Purpose:
The prototype of call back function when the memory data write completed event occurred.

Parameters:
i_lResult = The result of the writing.
i_Param = The pointer of user defined function or object that passed in by calling ADC_NewDataConnect.

Return:
None

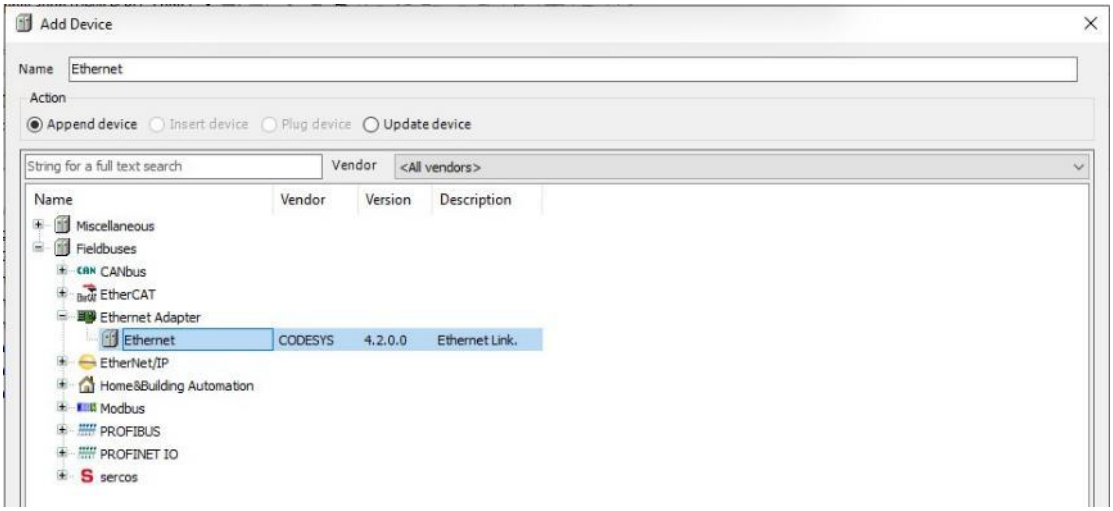
6. Start ADC

In addition, before using the ADC Library, users must create a Modbus TCP Server device in the Runtime environment and install the **adc-package** package on the Linux system.

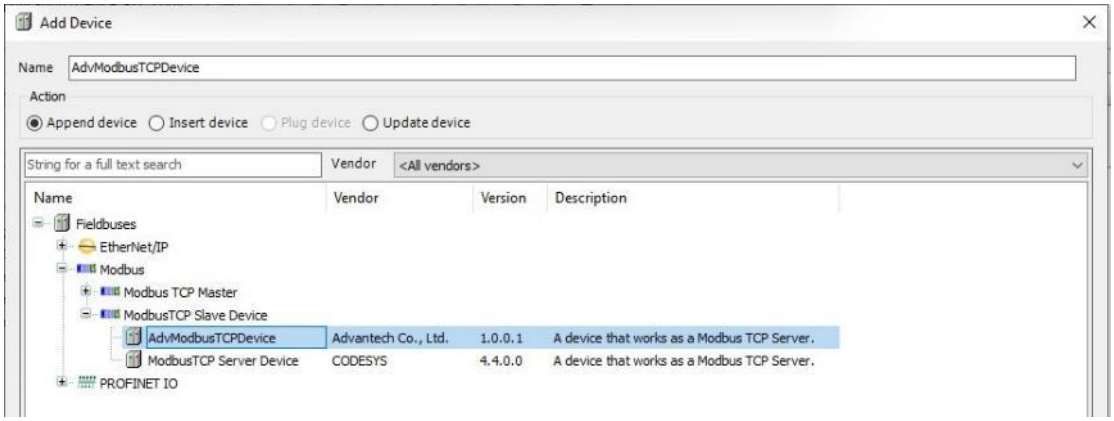
6.1. Create Modbus TCP Server

Note: The Modbus TCP Server device is required only when using the Linux version of the CODESYS Runtime.

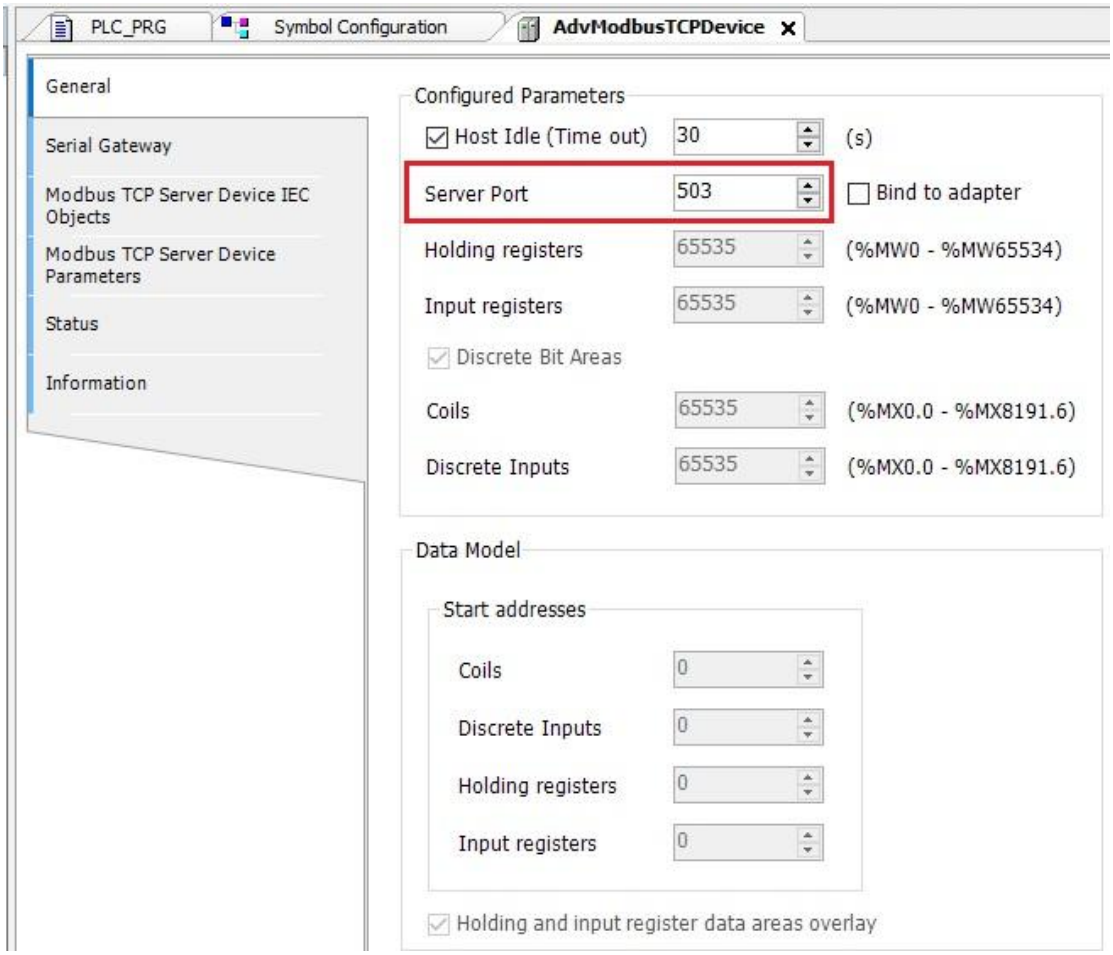
Step1: Add device “Ethernet”



Step2: Add device “AdvModbusTCPDevice” (Modbus TCP Server)



Step3: Configure “AdvModbusTCPDevice”. Set the server port to 503



Step4: Login and download project

6.2. adc-package Installation

The installation package follows the naming format:

adc-package_<version>_<linux-environment>.deb

For example, for installation on an x86 platform: adc-package_1.0.0.0_x86.deb

6.2.1. Installation

After copying the file to the system, run the following command to install:

sudo dpkg -i adc-package_1.0.0.0_x86.deb

```
oem@ubuntu:~$ sudo dpkg -i adc-package_1.0.0.0_x86.deb
Selecting previously unselected package adc-package.
(Reading database ... 190717 files and directories currently installed.)
Preparing to unpack adc-package_1.0.0.0_x86.deb ...
Unpacking adc-package (1.0.0.0) ...
Setting up adc-package (1.0.0.0) ...
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
/sbin/ldconfig.real: file /lib/libADCLibrary-x86.so is truncated
```

After installation:

The ADC-related shared libraries will be installed under /usr/lib.

```
oem@ubuntu:~$ ls -l /usr/lib/lib*
-rw-r--r-- 1 root root   459552 Nov  4 01:09 /usr/lib/libADCLibrary-x86.so
lrwxrwxrwx 1 root root       53 Oct 31 04:32 /usr/lib/libATAuxIO.so -> /usr/src/adv
lrwxrwxrwx 1 root root       52 Oct 31 04:32 /usr/lib/libATGPIO.so -> /usr/src/adva
lrwxrwxrwx 1 root root       53 Oct 31 04:32 /usr/lib/libATSMBUS.so -> /usr/src/adv
lrwxrwxrwx 1 root root       59 Oct 31 04:32 /usr/lib/libBoardResource.so -> /usr/s
-rw-r--r-- 1 root root 15646288 Nov  4 01:09 /usr/lib/libCmpPLCHandler-x86.so
-rwxr-xr-x 1 1001 root    27304 Sep 23 03:29 /usr/lib/libCmpVersion.so
lrwxrwxrwx 1 root root       54 Oct 31 04:32 /usr/lib/libEAPI-IPS.so -> /usr/src/ad
```

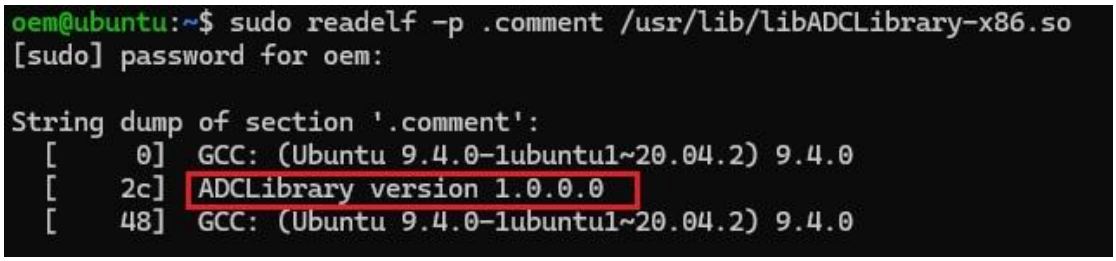
The ADC sample code will be installed under /usr/src/advantech/ADC/ExampleApp.

```
oem@oem-AMAX-5570:~$ ls /usr/src/advantech/ADC/ExampleApp/
ADCLibrary.h  Example1.cpp  Example2.cpp  Makefile
```

6.2.2. Check ADC Library Version

To check the installed ADC Library version, run:

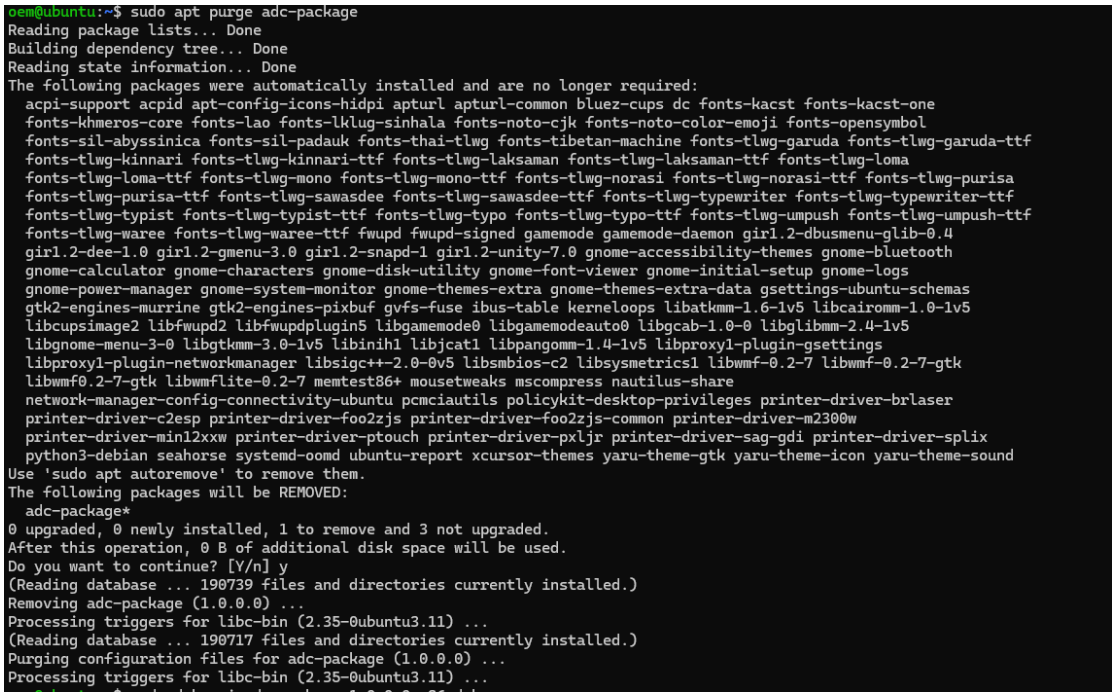
```
sudo readelf -p .comment /usr/lib/libADCLibrary-x86.so
```



6.2.3. Uninstallation

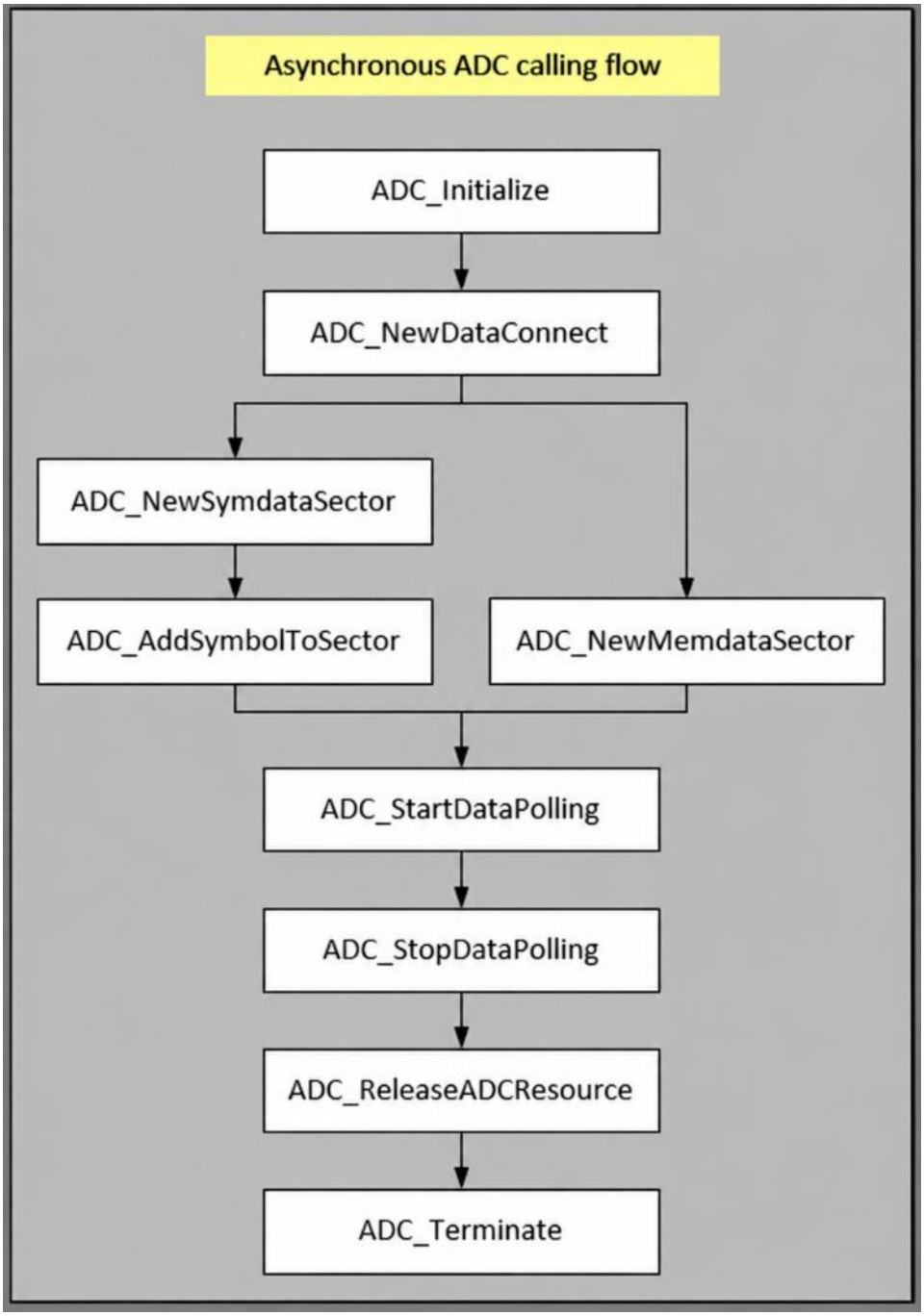
To uninstall the ADC package, run:

```
sudo apt purge adc-package
```

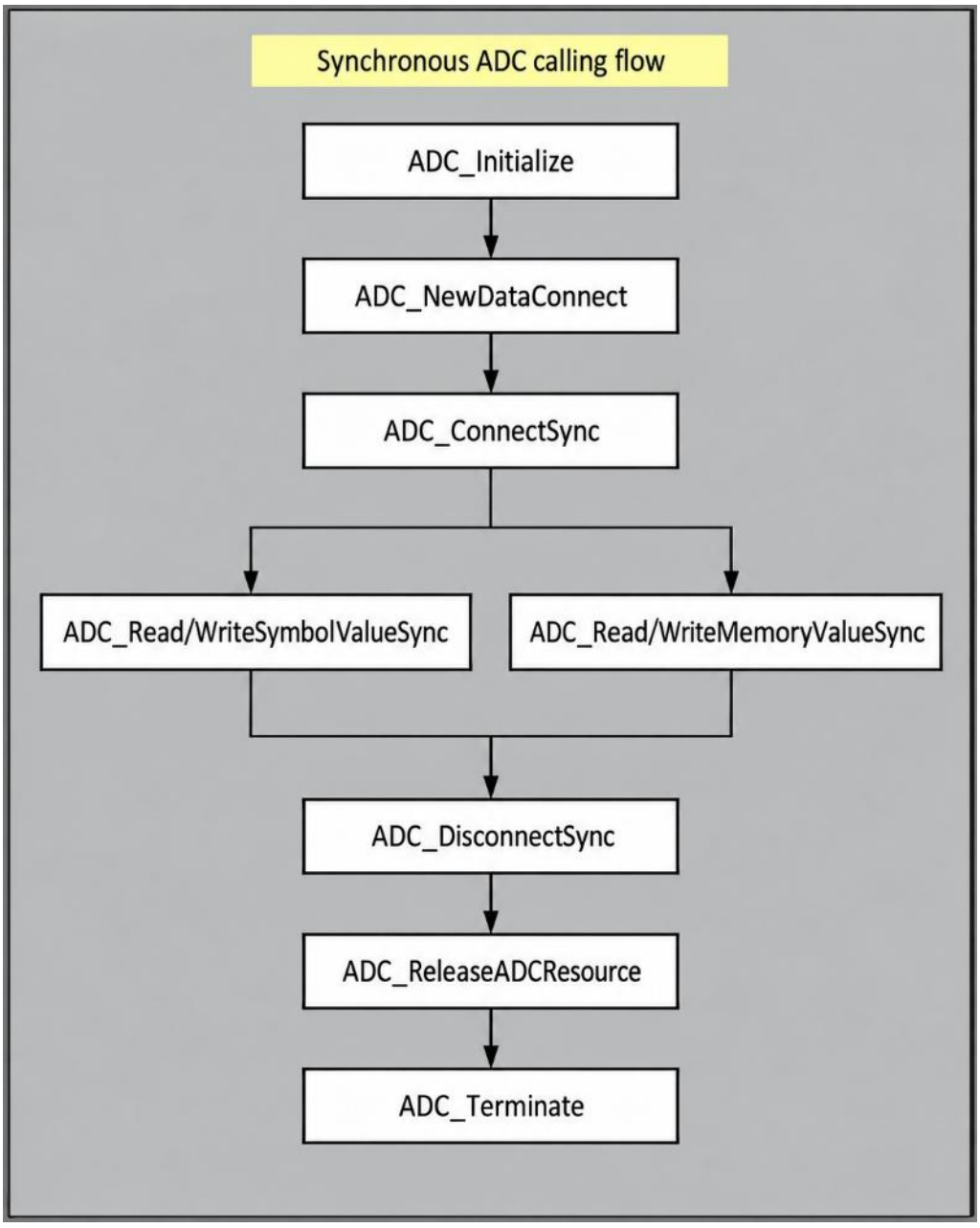


7. ADC calling flow

7.1. Asynchronous ADC calling flow



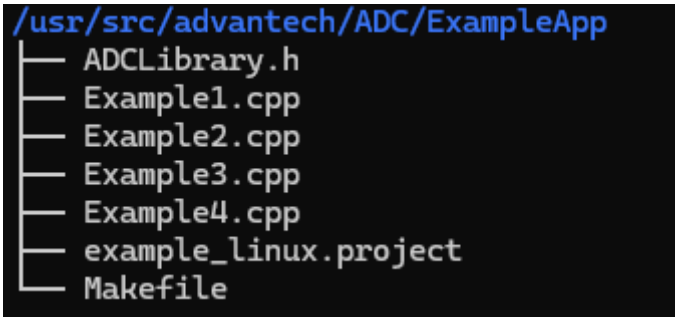
7.2. Synchronous ADC calling flow



The synchronous calling flow does not require Symbol sectors, Memory sectors, or ADC_StartDataPolling(). For polling-based access, continue to use the existing flow.

8. ADC sample code

After installing the adc-package, the ADC sample code and header file are installed under /usr/src/advantech/ADC/ExampleApp



ADCLibrary.h: Header file of the ADC Library

Example1.cpp: Demonstrates how to read and write Symbols and Flag Memory (%M) using asynchronous APIs.

Example2.cpp: Demonstrates how to read and write Local Flag Memory (%M).

Example3.cpp: Demonstrates how to read and write Symbols and Flag Memory (%M) using synchronous APIs.

Example4.cpp: Demonstrates how to read and write STRUCT variables using synchronous Symbol APIs.

Makefile: Builds the example applications supplied with this release.

example_linux.project: CODESYS project file for the example applications.

Note: After installation, the example source files are located in:

/usr/src/advantech/ADC/ExampleApp

You can copy the folder to another directory and compile it there to run the examples.

8.1. Settings and Build

8.1.1. Settings

Configure the target device's IP address and the login credentials used to access the Runtime environment.

Example1.cpp

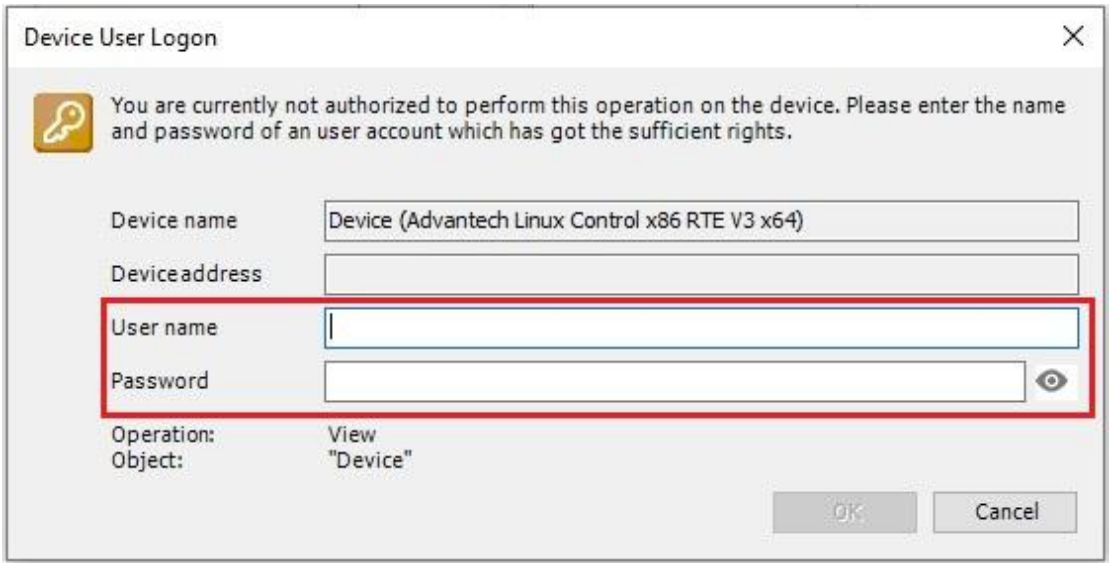
ExampleApp > Example1.cpp > main()

```
#include <stdio.h>
#include <unistd.h>

#include "ADCLibrary.h"

// Mutable, persistent buffers for APIs declared with char* parameters.
// Keep these alive while asynchronous polling and callbacks may use them.
char connectionName[] = "CONN1";
char plcIp[] = "10.0.0.1";
char username[] = "root";
char password[] = "password";
char symbolSectorName[] = "SymSec";
char memorySectorName[] = "MemSec";
char symbolName1[] = "Application.PLC_PRG.sym1";
char symbolName2[] = "Application.PLC_PRG.sym2";
char symbolName3[] = "Application.PLC_PRG.sym3";
char memoryWordAddress[] = "%MW0";
```

Note: Set the username and password for logging in to the CODESYS Runtime.



8.1.2. Build Sample Code

Navigate to the **ExampleApp** directory and execute the following command:

```
make clean && make
```

Note: The Makefile automatically detects the host CPU architecture and selects the appropriate build configuration. No additional CPU parameters are required for native compilation.

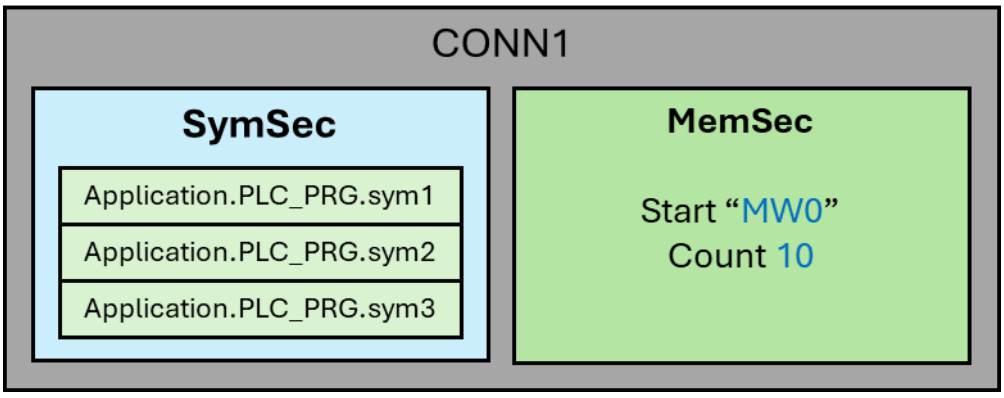
To build for a specific CPU architecture (for example, ARM 64-bit):

```
make clean && make CPU=arm64
```

8.2. Basic example

Please reference the sample code **Example1**. This example shows how to use ADC API to access controller symbols and flag memory.

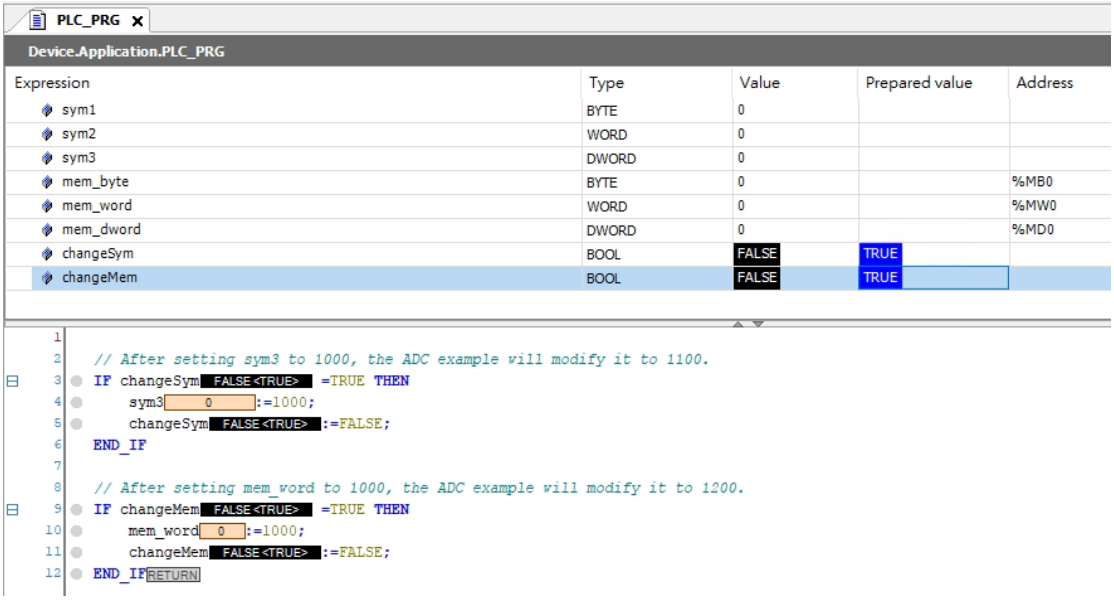
This example creates a new data connection named "**CONN1**". And then create a symbol sector named "**SymSec**" to "**CONN1**". Add 3 symbols to the sector. Then create a memory sector named "**MemSec**" to "**CONN1**". Set the sector from **%MW0** ~ **%MW9**. As shown below.



After calling **ADC_StartDataPolling()**, the ADC thread will start polling all controllers and their symbol / memory data. In the callback read complete function, check whether sym3 is

completed and the value is 1000, modify its value to 1100 and write it back to the controller. Similarly, when reading% MW0 is completed and the value is 1000, modify the value to 1200 and write it back to the controller.

Back to the CODESYS project, we set **changeSym** and **changeMem** to **true**. Then, the ADC example will modify the value of **sym3** and **%MW0**.



Note: To access memory through Symbols, configure variables and symbols as described in Section 2.3.

In the example application:

- ***Add Application.PLC_PRG.sym_mem under the SymSec sector.***
- ***Configure MemSec to map the appropriate %M address according to the variable definition.***

After completing these settings, the APIs ADC_ReadSymbolValue and ADC_WriteSymbolValue can be used to read and write memory through Symbols.

8.3. Access local flag memory

Please reference the sample code **Example2**. This example shows how to use ADC API to access local flag memory.

If you want to access the local flag memory (must be the same device as the CODESYS RTE), you can call read/write APIs (**ADC_ReadSingleMemoryValue_Local** / **ADC_ReadMultipleMemoryValue_Local** / **ADC_WriteSingleMemoryValue_Local** / **ADC_WriteMultipleMemoryValue_Local**) directly without create data connection and sector. And after the initialization is successful, it can be accessed directly without calling **ADC_StartDataPolling**.

This example first reads %MW0 from the local flag memory. If the result is successful, check if the value is 1000, and then write back 1200 to %MW0.

Then read the entire flag memory data and write them all to 0xff. (Need to define "ReadWriteAll")

8.4. Synchronous access

Please reference the sample code Example3. This example demonstrates synchronous Symbol and Memory read/write, including retrieving the Symbol list using ADC_GetSymbolList.

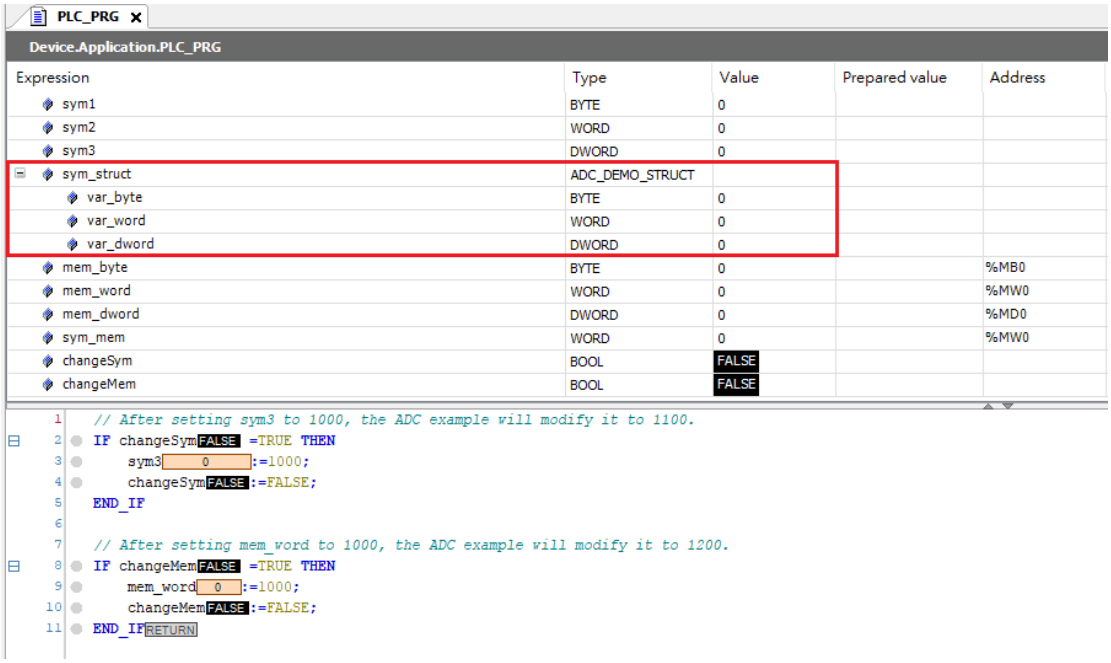
The example reads and writes BYTE, WORD and DWORD Symbols, as well as %MW0 (16-bit) and %MD1 (32-bit) Memory values. It also demonstrates setting the Memory connection port and releasing ADC resources.

Note: Configure the PLC IP, credentials and target Symbols for your device. This example actively overwrites writable Symbols of the supported types and the two specified Memory locations; run it only against a safe test project.

8.5. STRUCT access

Please reference the sample code Example4. This example demonstrates synchronous read/write of a CODESYS STRUCT Symbol (Application.PLC_PRG.sym_struct).

The example uses ADC_DEMO_STRUCT with BYTE, WORD and DWORD fields, writes the complete structure and reads it back. The PLC and C++ structure layouts must match (8 bytes, including padding).



Note: The PLC Symbol must be published and writable. Struct layout must be verified on the target PLC; a mismatch can cause incorrect field values even when the supplied buffer length is 8 bytes.

9. ADC library error code

Error Code		
Code	Name	Description
0	ADC_SUCCESS	Action successful.
1001	ADC_HANDLE_INVALID	The connection handle is invalid. Typically the wrong connection name.
1002	ADC_PARAMETER_INVALID	At least one parameter is invalid.
1003	ADC_THREAD_RUNNING	The ADC thread is running.
1004	ADC_THREAD_STOP	The ADC thread is stop.
1005	ADC_MEMALLOC_FAILED	Failed to allocate memory.
1006	ADC_OBJECT_DUPLICATED	The specified object already exists.
1007	ADC_OBJECT_NOT_EXIST	The specified object does not exist.
1008	ADC_SETPRIO_FAILED	Failed to set the ADC thread priority.
1009	ADC_CREATE_THREAD_FAILED	Failed to create the ADC connection.
1010	ADC_EXCEED_SECTOR_SIZE	The total size of the memory item exceeds the sector size.
1011	ADC_BUFFER_TOO_SMALL	The provided buffer is too small.
1012	ADC_SIZE_MISMATCH	The data size does not match the Symbol size.
1013	ADC_INCOMPLETE_DATA	The data returned for the operation is incomplete.
1014	ADC_WRONG_CONTEXT	The API cannot be called from the current communication thread context.
2001	ADC_PLC_CONNECTED	PLC is connected, symbols are loaded and all of them are verified. Now you can work correctly with the ADC symbol object.
2002	ADC_PLC_NOT_CONNECTED	PLC is not connected.
2003	ADC_PLC_NO_SYMBOLS	No symbols are available or symbol mismatch because of changed project.

2004	ADC_PLC_SYMBOLS_LOADED	PLC is connected and symbols are loaded.
2005	ADC_PLC_DISCONNECT	Connection is just getting terminated.
2006	ADC_PLC_NO_CONFIGURATION	Configuration of the ADC is invalid.
2007	ADC_PLC_NOT_CONNECTED_SYMBOLS_LOADED	PLC is currently not available, but the symbol information could be loaded offline. The ADC tries further to connect to the PLC.
2008	ADC_PLC_SYMBOL_NOT_MATCH	The symbol sector contains invalid elements or the symbol sector does not match to the actual symbol list.
2009	ADC_PLC_SYMBOL_SIZE_NOT_MATCH	The variable size is less than the actual symbol size
2010	ADC_PLC_SYMBOL_VALUE_NOT_READY	Symbol sector was defined and ADC thread is running, but the first read of the symbol sector has not yet been finished. Try again later to read the variables.
2011	ADC_PLC_NOT_VALID_DEVICE	Target device is not Advantech RTE.
2012	ADC_PLC_SYMBOL_LOAD_FAILED	Failed to load PLC Symbols.
3001	ADC_MEM_CONNECTED	Connect to the ADC server successes.
3002	ADC_MEM_WSASTART_FAILED	Failed to initial ADC resource.
3003	ADC_MEM_WSACLEAN_FAILED	Failed to terminate ADC resource
3004	ADC_MEM_CREATESOCK_FAILED	Failed to create socket to connect to ADC server.
3005	ADC_MEM_WAIT_TIMEOUT	Timeout exceeded.
3006	ADC_MEM_CONNECT_FAILED	Failed to connect to the ADC server.
3007	ADC_MEM_TCPSEND_FAILED	Failed to send packet to ADC server.
3008	ADC_MEM_TCPRECV_FAILED	Failed to receive packet from ADC server
3009	ADC_MEM_SOCKET_CLOSED	The connection does not connect.
3010	ADC_MEM_THREAD_BUSY	The previous write memory operation was not completed
3901	ADC_MEM_MODEXCEPT_01	Illegal function
3902	ADC_MEM_MODEXCEPT_02	Illegal data address
3903	ADC_MEM_MODEXCEPT_03	Illegal data value

3904	ADC_MEM_MODEXCEPT_04	Slave device failure
3905	ADC_MEM_MODEXCEPT_05	Acknowledge
3906	ADC_MEM_MODEXCEPT_06	Slave device busy
3907	ADC_MEM_MODEXCEPT_07	Negative acknowledge
3908	ADC_MEM_MODEXCEPT_08	Memory parity error
3999	ADC_MEM_MODEXCEPT_99	Invalid packet

Troubleshooting

1. PLC channel information

When you have ruled out the network problem but still cannot connect to RTE through ADC library. You can check whether all communication channels are occupied.

Open the **CODESYS Project** and then choose the **PLC Shell** tab. Enter the **channelinfo** command to check whether there are still channels displayed as FREE. If none of them appear as FREE, please try to close unnecessary connections. Each ADC connection will occupy a channel.

